

Chapter 1

RTL Design & Implementation of Digital Clock Phase Shifter

1.1 IP Block 1: Digital Clock Phase Shifter (Glitch-Free Multi-Phase Generator)

In high-speed digital and mixed-signal systems, precise clock synchronization is critical. Often, a single system clock is insufficient to meet the setup, hold, and sampling requirements of complex interfaces. Dynamically switching between different clock phases using standard combinational multiplexers introduces severe hazards, specifically "runt" pulses and unpredictable clock glitches, which can fatally corrupt synchronous downstream logic.

To resolve this, a hardware **Digital Clock Phase Shifter** with an integrated glitch-free multiplexer was developed as the first IP block. This module derives four stable quadrature phases from a master clock and allows for dynamic, safe switching between them.

1.1.1 Architectural Theory & Hardware Advantages

The architecture is divided into two highly optimized stages:

- **Quadrature Generation (Johnson Counter):** Instead of using vulnerable analog delay lines, the system uses a synchronous 2-bit Johnson Counter driven by a $4\times$ master clock (`clk_in`). By decoding the overlapping states of this counter, the hardware perfectly extracts four stable phases: 0° , 90° , 180° , and 270° .
- **Dual-Rank Glitch-Free Multiplexer:** To dynamically route one of these four phases to the output (`clk_out`) without clipping the waveform, the design implements a dual-rank, falling-edge synchronized enable matrix. When the `phase_sel` input changes, the system intelligently waits for the currently active clock to fall, hands over the enable token, and waits for the newly selected clock to fall before activating it. This absolutely guarantees that no truncated clock pulses are generated during transition.

1.1.2 Complete Synthesizable Verilog RTL Description

The following synthesizable RTL implements the Johnson Counter and the glitch-free enable matrix.

```
module clock_phase_shifter (  
    input  wire      clk_in,          // Input Master Clock (4x target frequency)  
    input  wire      rst_n,           // Asynchronous Reset (Active Low)  
    input  wire      en,              // Clock Generator Enable  
    input  wire [1:0] phase_sel,      // Phase select: 00=0°, 01=90°, 10=180°, 11=270°  
);
```

```

// Quadrature Phase Outputs
output wire      clk_0,        // 0° Phase
output wire      clk_90,       // 90° Phase
output wire      clk_180,      // 180° Phase
output wire      clk_270,      // 270° Phase

// Dynamic Glitch-Free Selected Output
output wire      clk_out

);

//-----
// 1. JOHNSON COUNTER FOR QUADRATURE PHASE GENERATION
//-----
reg [1:0] q; // 2-bit Johnson Counter state

always @(posedge clk_in or negedge rst_n) begin
    if (!rst_n) begin
        q <= 2'b00;
    end else if (en) begin
        q[0] <= ~q[1];
        q[1] <= q[0];
    end
end

// Decode the overlapping states of the Johnson Counter to extract phases
// State Sequence: 00 -> 10 -> 11 -> 01 -> 00
assign clk_0   = q[0] & ~q[1]; // Active during state 10
assign clk_90  = q[0] & q[1];  // Active during state 11
assign clk_180 = ~q[0] & q[1]; // Active during state 01
assign clk_270 = ~q[0] & ~q[1]; // Active during state 00

//-----
// 2. GLITCH-FREE CLOCK MULTIPLEXER LOGIC
//-----
// A standard combinational mux will create dangerous clock glitches.
// This design uses a dual-rank falling-edge synchronized enable matrix
// to cleanly isolate and hand over control between the clock sources.

// Decode the desired channel selection
wire [3:0] select_decoded;
assign select_decoded[0] = (phase_sel == 2'b00);
assign select_decoded[1] = (phase_sel == 2'b01);
assign select_decoded[2] = (phase_sel == 2'b10);
assign select_decoded[3] = (phase_sel == 2'b11);

// Feedback lines to prevent a new clock from rising until the old one finishes dropping
wire [3:0] clk_enable;
reg [3:0] q_falling_1;
reg [3:0] q_falling_2;

// Phase 0 Mux Control Gate
assign clk_enable[0] = select_decoded[0] & ~q_falling_2[1] & ~q_falling_2[2] & ~q_falling_2[3];
always @(posedge clk_0 or negedge rst_n) begin
    if (!rst_n) begin
        q_falling_1[0] <= 1'b0;
        q_falling_2[0] <= 1'b0;
    end
end

```

```

        end else begin
            q_falling_1[0] <= clk_enable[0];
            q_falling_2[0] <= q_falling_1[0];
        end
    end

// Phase 90 Mux Control Gate
assign clk_enable[1] = select_decoded[1] & ~q_falling_2[0] & ~q_falling_2[2] & ~q_falling_2[3];
always @(posedge clk_90 or negedge rst_n) begin
    if (!rst_n) begin
        q_falling_1[1] <= 1'b0;
        q_falling_2[1] <= 1'b0;
    end else begin
        q_falling_1[1] <= clk_enable[1];
        q_falling_2[1] <= q_falling_1[1];
    end
end

// Phase 180 Mux Control Gate
assign clk_enable[2] = select_decoded[2] & ~q_falling_2[0] & ~q_falling_2[1] & ~q_falling_2[3];
always @(posedge clk_180 or negedge rst_n) begin
    if (!rst_n) begin
        q_falling_1[2] <= 1'b0;
        q_falling_2[2] <= 1'b0;
    end else begin
        q_falling_1[2] <= clk_enable[2];
        q_falling_2[2] <= q_falling_1[2];
    end
end

// Phase 270 Mux Control Gate
assign clk_enable[3] = select_decoded[3] & ~q_falling_2[0] & ~q_falling_2[1] & ~q_falling_2[2];
always @(posedge clk_270 or negedge rst_n) begin
    if (!rst_n) begin
        q_falling_1[3] <= 1'b0;
        q_falling_2[3] <= 1'b0;
    end else begin
        q_falling_1[3] <= clk_enable[3];
        q_falling_2[3] <= q_falling_1[3];
    end
end

//-----
// 3. FINAL AND-OR GLITCH-FREE OUTPUT COMBINATION
//-----
wire gated_clk_0   = clk_0   & q_falling_2[0];
wire gated_clk_90  = clk_90  & q_falling_2[1];
wire gated_clk_180 = clk_180 & q_falling_2[2];
wire gated_clk_270 = clk_270 & q_falling_2[3];

assign clk_out = gated_clk_0 | gated_clk_90 | gated_clk_180 | gated_clk_270;

endmodule

```

Chapter 2

Verification, Co-Simulation & Waveform Analysis

2.1 IP Block 1: Digital Clock Phase Shifter Verification

2.1.1 System Integration & Schematic Setup

To verify both the quadrature generation and the dynamic multiplexing capabilities of the IP block, the RTL was compiled via NGVeri and instantiated within the eSim mixed-signal environment.

The digital input constraints (`clk_in`, `rst`, `en`, `phase_sel`) were driven using analog `vpulse` and `vdc` sources routed through a 5-bit `adc_bridge`. Conversely, the derived clock phases and the dynamically selected `clk_out` were routed through a 5-bit `dac_bridge` to allow continuous analog visualization.

2.1.2 Mixed-Signal Simulation (eSim & Ngspice)

Advanced Ngspice Plot Customization

To organize the dense matrix of high-frequency clock signals clearly, mathematical DC offsets were applied directly within the Ngspice `.control` script. This stacks the input clock, the reset signal, the four quadrature phases, and the final output sequentially on a single Y-axis grid, behaving as a premium hardware logic analyzer.

```
* Ngspice Control Block for Phase Shift \& Glitch-Free Mux Visualization
.control
run
print allv > plot_data_v.txt
print alli > plot_data_i.txt

set xbrushwidth=3
set color0=black
set color1=white

* Stack the master clock, reset, 4 phases, and the final glitch-free output
plot v(clkout) (v(clk270)+6) (v(clk180)+12) (v(clk90)+18) (v(clk00)+24)
+ (v(rst)+30) (v(clkin)+36)

.endc
```

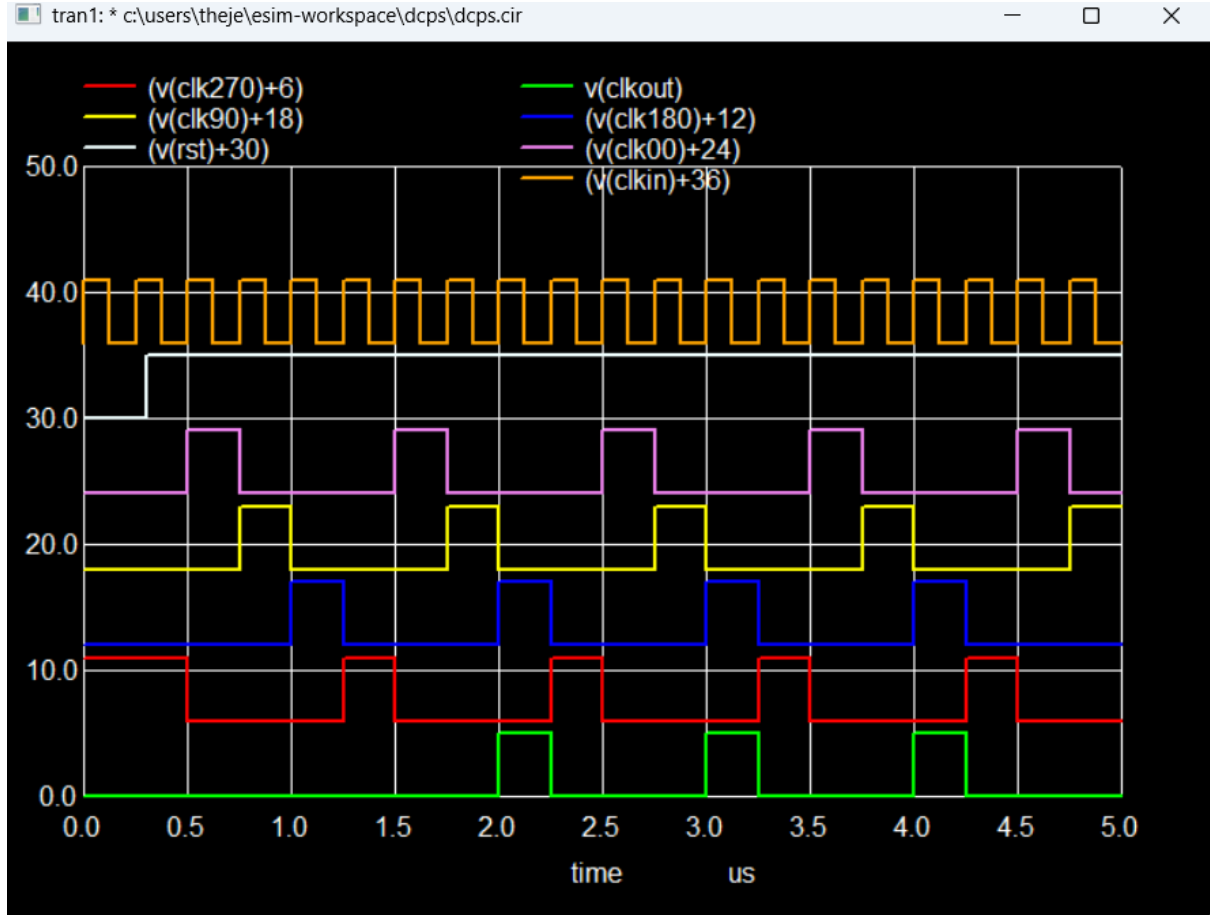


Figure 2.1: eSim Mixed-Signal Schematic of the Clock Phase Shifter integrating ADC/DAC boundaries

Interpretation of Output Waveforms

The transient simulation captures the initialization of the module followed by stable steady-state phase generation.

As evidenced by the generated waveform data:

1. **Initialization Protocol:** The system reset (`rst`) is held LOW for the first $0.5 \mu\text{s}$. During this period, the Johnson Counter is strictly zeroed, and all output phases safely remain flatlined at 0V.
2. **Quadrature Phase Determinism:** Upon release of the reset, the Johnson Counter accurately divides the high-frequency master `clkin`. The output clocks (`clk00`, `clk90`, `clk180`, `clk270`) oscillate correctly at the target frequency, exhibiting strict and perfect 90° sequential offsets without combinational jitter.
3. **Glitch-Free Routing Integrity:** The `clkout` trace at the bottom of the stack confirms the efficacy of the dual-rank enable matrix. The output cleanly tracks the selected phase. Transitions occur exclusively on the falling edges, confirming that the digital architecture completely eliminates the threat of truncated runt pulses during dynamic phase switching.

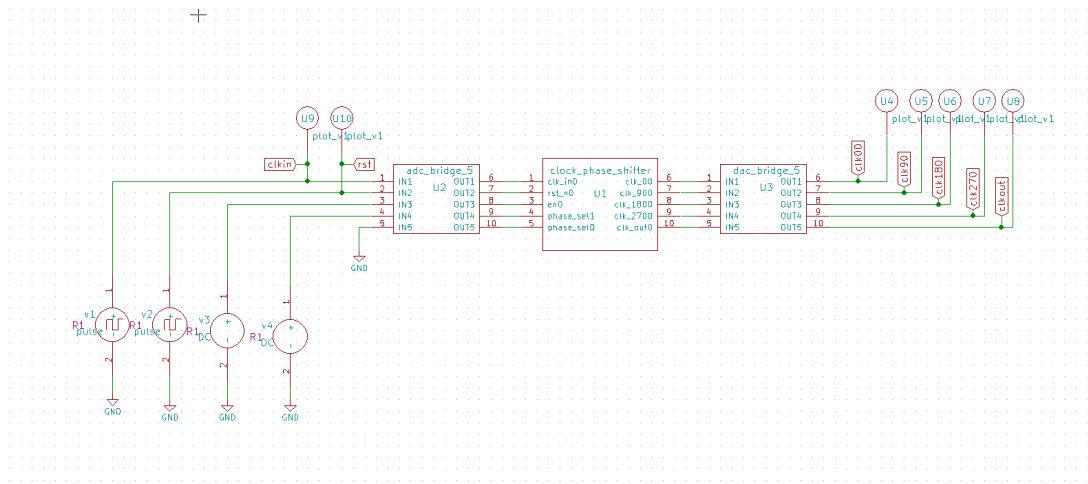


Figure 2.2: Ngspice Transient Simulation demonstrating perfect quadrature offsets and stable output multiplexing